



UNIVERSIDAD TECNOLÓGICA DE PANAMÁ

FACULTAD DE INGENIERÍA DE SISTEMAS COMPUTACIONALES

DEPARTAMENTO DE COMPUTACIÓN Y SIMULACIÓN DE SISTEMAS

LICENCIATURA EN INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

Asignatura:

Sistemas Operativos

Asignación:

Laboratorio #1: Preparación del Entorno Linux para Sistemas Operativos

Profesor:

Prof. Isacio Tamayo

Estudiante:

James Tack

Grupo:

1IL143

Fecha de entrega:

4 de septiembre de 2026

2do Semestre 2026

De qué se trata el laboratorio

El objetivo es dejar montada la máquina virtual Linux que se va a usar el resto del semestre, entrar a ella por SSH y comprobar que trae las herramientas que hacen falta: gcc, make, strace y htop, más un programa compilado llamado `test_proc` que sirve de prueba de humo.

La guía deja elegir entre dos caminos para desplegar la VM. La opción A usa Terraform y la opción B usa Vagrant. Intenté primero la A porque el paso del plan aparece marcado como obligatorio, y terminé completando el laboratorio con la B porque el proveedor de Terraform que pide el archivo `main.tf` no se lleva con la versión de VirtualBox que tengo instalada. Los dos intentos están documentados: el que falló también enseña algo.

Paso 1. Revisar las herramientas del anfitrión

El anfitrión es un Dell Inspiron 16 5645 con Windows 11 Pro. VirtualBox ya estaba de la tarea anterior; Vagrant y Terraform los instalé con el gestor de paquetes de Windows:

```
winget install --id Hashicorp.Vagrant -e
winget install --id Hashicorp.Terraform -e
```

Después comprobé las tres versiones de una vez. La guía pide VirtualBox 6.1 o superior y aquí hay 7.2.16, así que por ese lado no hay problema (o eso creía yo en ese momento).

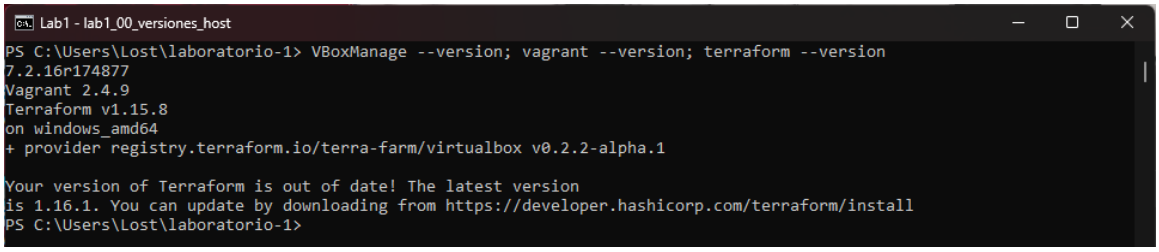


Figura 1. Versiones de VirtualBox, Vagrant y Terraform en el sistema anfitrión.

Paso 2. Abrir la carpeta del laboratorio

Descargué la carpeta `lab1-preparacion-entorno` del repositorio del curso y la dejé en `C:\Users\Lost\laboratorio-1`. La guía usa `ls -la`, que es de bash; en PowerShell el equivalente es `Get-ChildItem -Force`.

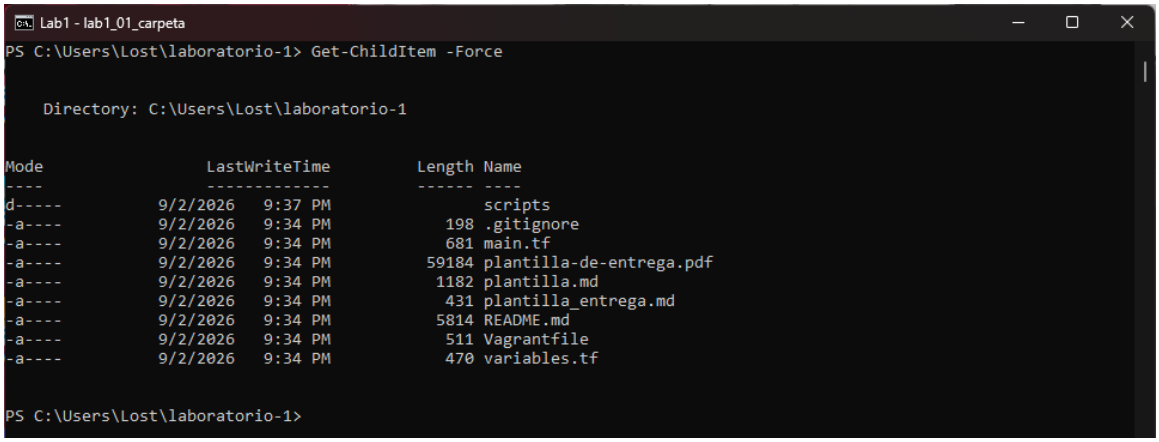


Figura 2. Contenido de la carpeta del laboratorio antes de desplegar nada.

Están los tres archivos de Terraform (`main.tf`, `variables.tf`, `outputs.tf` no vino en el repositorio), el `Vagrantfile`, la plantilla de entrega y la carpeta `scripts` con el aprovisionamiento. El `.box` local del que habla la guía no me hizo falta: ninguna de las dos opciones lo necesita si hay internet, porque la imagen se descarga sola.

Opción A. Despliegue con Terraform

Terraform describe la infraestructura en archivos de texto y se encarga de crearla. El `main.tf` del laboratorio declara un recurso `virtualbox_vm` con dos CPU, 2048 MiB de RAM, una imagen de Ubuntu Bionic descargada de Vagrant Cloud y un adaptador de red de tipo hostonly.

Paso 3. terraform init

Este comando lee el bloque `required_providers` y descarga el plugin que sabe hablar con VirtualBox. En este caso es `terra-farm/virtualbox` en su versión 0.2.2-alpha.1, que está fijada en el archivo.

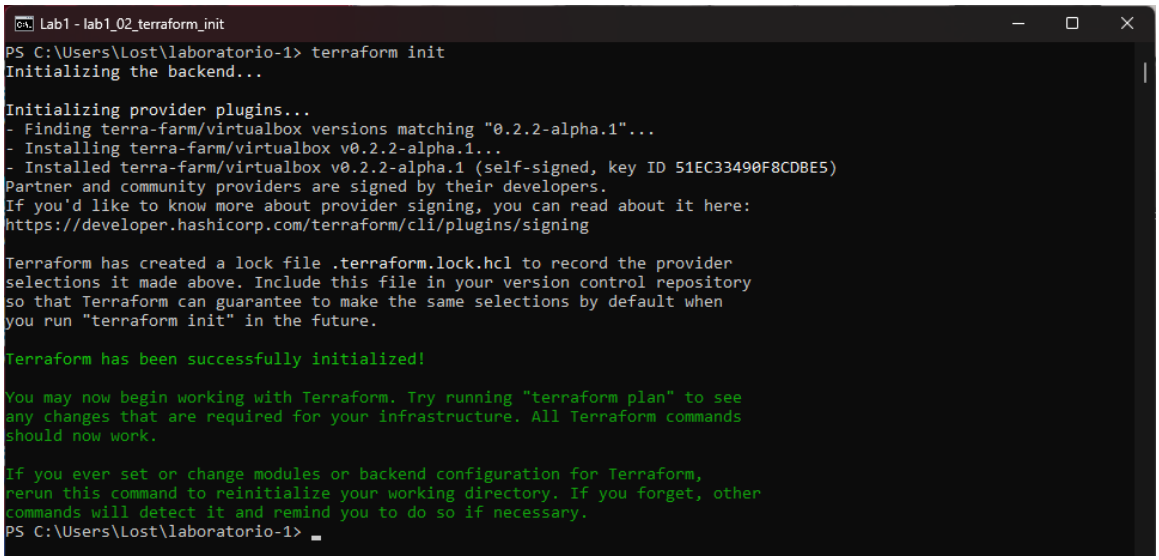


Figura 3. terraform init descarga el proveedor terra-farm/virtualbox y crea el archivo de bloqueo.

Vale la pena fijarse en dos cosas de esa salida. El proveedor está firmado por su autor y no por HashiCorp, y su número de versión dice alpha. Las dos son señales de que no es código de producción. Más adelante se entiende por qué lo menciono.

Paso 4. terraform plan

El plan es la parte que la guía marca como obligatoria, y con razón: muestra exactamente qué se va a crear antes de tocar nada. Terraform compara lo que dicen los archivos contra lo que existe y devuelve la diferencia.

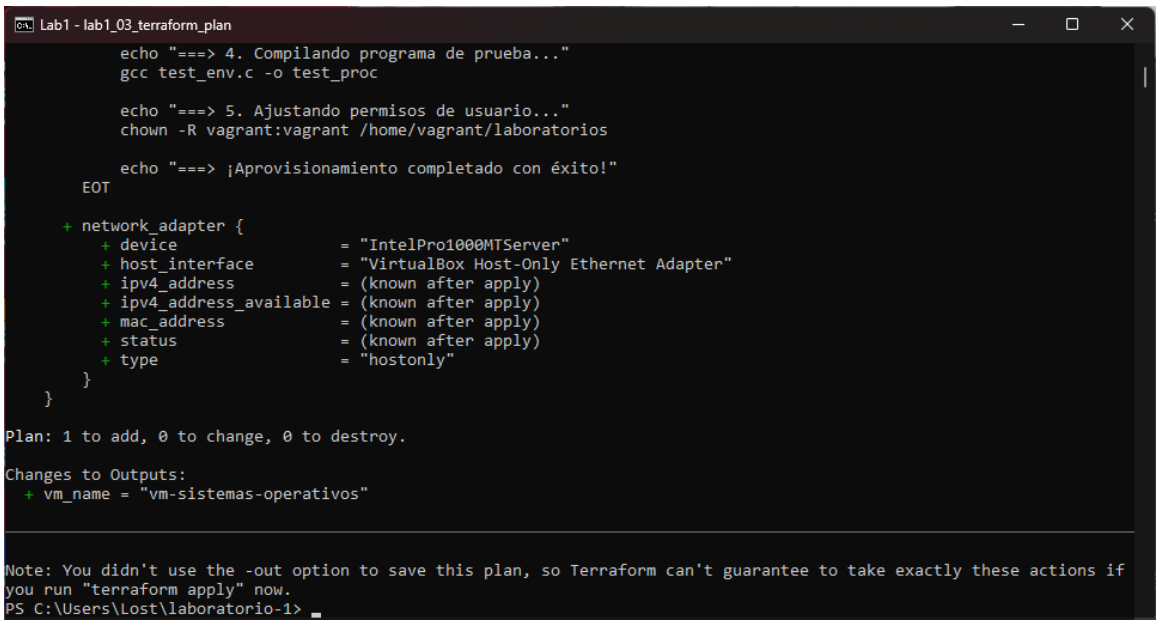


Figura 4. Cierre de terraform plan: un recurso a crear, ninguno a modificar ni a destruir.

La línea que resume todo es `Plan: 1 to add, 0 to change, 0 to destroy`. Arriba de ella aparece el script de aprovisionamiento completo, que Terraform va a pasarle a la VM como `user_data`, y el bloque del adaptador de red con los campos que todavía no puede saber, marcados como `(known after apply)`. Uno de esos campos es la dirección IPv4, y ahí está el problema que venía en camino.

Paso 5. terraform apply

Con el plan revisado, toca aplicarlo:

```
terraform apply -auto-approve
```

La primera vez ni siquiera llegó a crear la VM: el proveedor invoca a `VBoxManage` y no lo encontraba, porque el instalador de VirtualBox no agrega su carpeta al PATH de Windows. Lo arreglé añadiendo `C:\Program Files\Oracle\VirtualBox` al PATH del usuario y volví a intentarlo. Esta vez sí creó la máquina, la arrancó, y se quedó esperando.

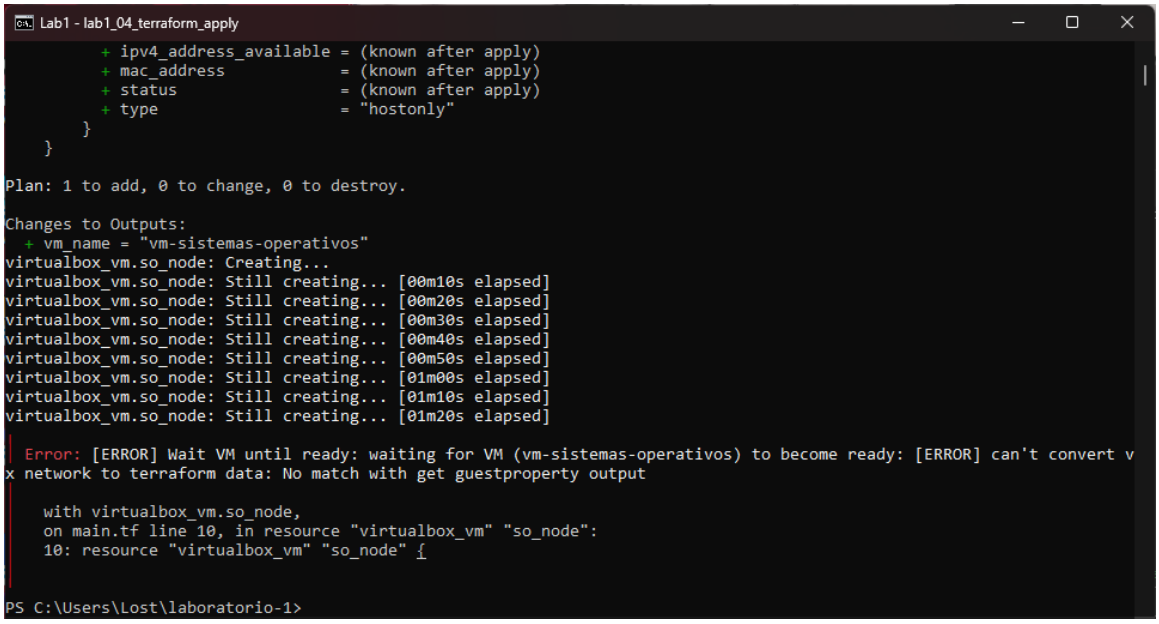


Figura 5. terraform apply crea la VM, espera un minuto y medio y se rinde con un error de red.

El error dice: *can't convert vbox network to terraform data: No match with get guestproperty output*.

Por qué falla

Traducido: el proveedor arrancó la máquina y se puso a preguntarle a VirtualBox cuál es la IP del huésped, para poder rellenar ese campo `ipv4_address` que quedó pendiente en el plan. Esa consulta se hace con `VBoxManage guestproperty enumerate`, y el que escribe esas propiedades desde adentro es el servicio de las Guest Additions. Si el servicio no está corriendo, la propiedad no existe y el proveedor se queda esperando algo que nunca va a llegar.

Lo comprobé mientras el apply seguía dando vueltas:

```
C:\> VBoxManage showvminfo "vm-sistemas-operativos" --machinereadable | findstr
GuestAdditions
GuestAdditionsRunLevel=0

C:\> VBoxManage guestproperty enumerate "vm-sistemas-operativos"
/VirtualBox/HostInfo/VBoxVer = '7.2.16' ...
(no aparece ninguna propiedad /VirtualBox/GuestInfo/Net/...)
```

Nivel 0 es servicio detenido. Y al mirar la pantalla de la máquina se ve que el arranque se quedó clavado:

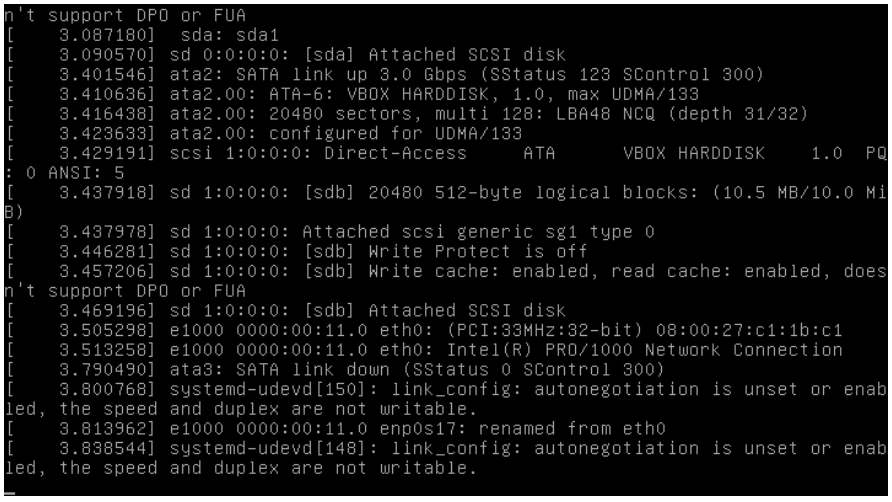


Figura 6. Pantalla de la VM creada por Terraform: el arranque se detiene y nunca llega al login.

Son dos problemas encima del mismo diseño. Las Guest Additions que trae esa imagen son de la época de VirtualBox 6.x y no compilan contra 7.2, así que el servicio no levanta. Y el `main.tf` le pone a la VM un único adaptador de tipo hostonly, sin NAT, o sea sin salida a internet: aunque la máquina hubiera arrancado bien, el `apt-get update` del script de aprovisionamiento tampoco habría podido descargar nada.

Así que la opción A no es cuestión de haberla ejecutado mal. El proveedor tiene cinco años, sigue en alpha y no se actualizó para VirtualBox 7.

```
C:\> VBoxManage controlvm "vm-sistemas-operativos" poweroff
C:\> VBoxManage unregistervm "vm-sistemas-operativos" --delete
```

Opción B. Despliegue con Vagrant

Vagrant hace lo mismo pero por otro camino. En vez de hablar con la API de VirtualBox, usa una imagen ya preparada (una *box*), la clona, le configura una red NAT con redirección de puertos para el SSH y corre el script de aprovisionamiento por esa misma conexión. No depende de las Guest Additions para nada de eso.

Paso 6. vagrant up

El `Vagrantfile` del laboratorio pide la imagen `ubuntu/bionic64`, le pone de nombre `vm-sistemas-operativos`, hostname `so-environment`, 2048 MB de RAM, 2 CPU y ejecuta `scripts/setup_env.sh` al final. Un solo comando:

```
vagrant up
```

Tardó bastante. Primero descarga la imagen, que son unos 500 MB, después la importa, arranca la máquina, espera al SSH y recién ahí empieza el aprovisionamiento: `apt-get update`, instalar el `build-essential` entero con gcc, make, gdb, htop, strace, tree, lsof y compañía, crear `~/laboratorios/unidad1`, escribir `test_env.c` y compilarlo.

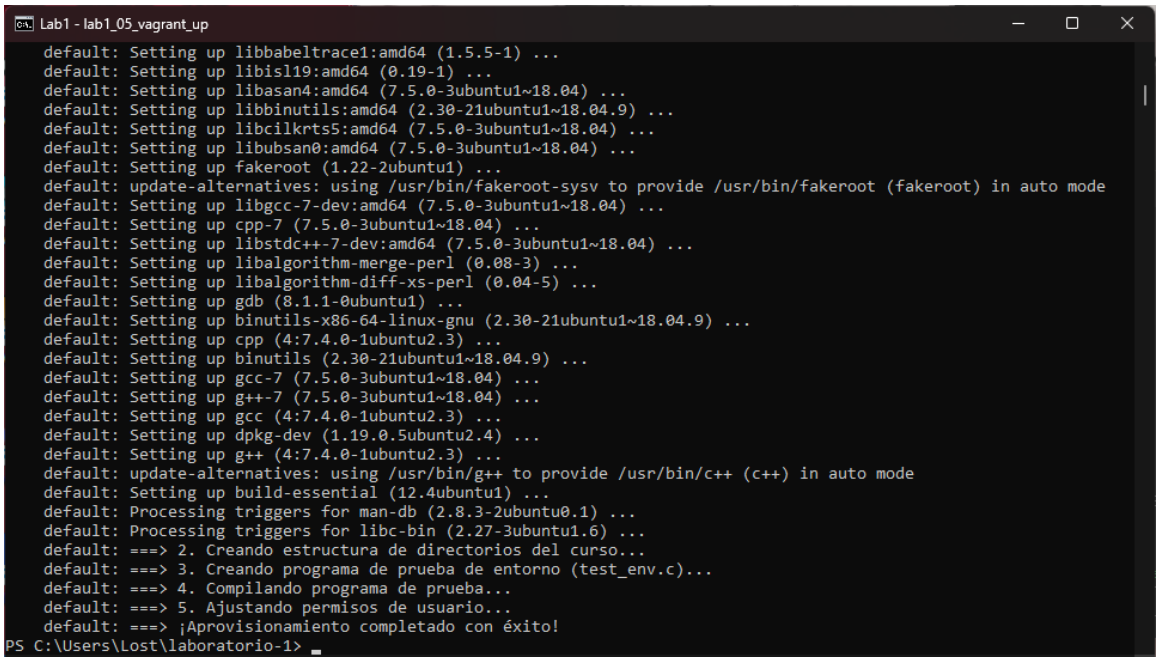


Figura 7. Final de vagrant up: se ven los paquetes instalándose y los cinco pasos del script de aprovisionamiento.

La última línea es la que importa: `===> ¡Aprovisionamiento completado con éxito!`. Los cinco pasos numerados del script salieron todos.

Paso 7. Comprobar que la VM quedó corriendo

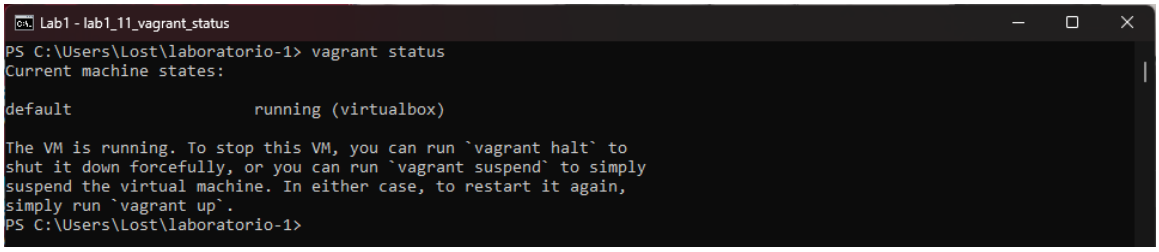


Figura 8. vagrant status confirma que la máquina default está en estado running.

Acceso por SSH

Paso 8. Entrar a la máquina

Desde la misma carpeta:

```
vagrant ssh
```

No pide contraseña ni hay que saberse ninguna IP, porque Vagrant guarda la llave privada y el puerto cuando crea la máquina. Ya adentro, lo primero fue confirmar dónde estaba parado.

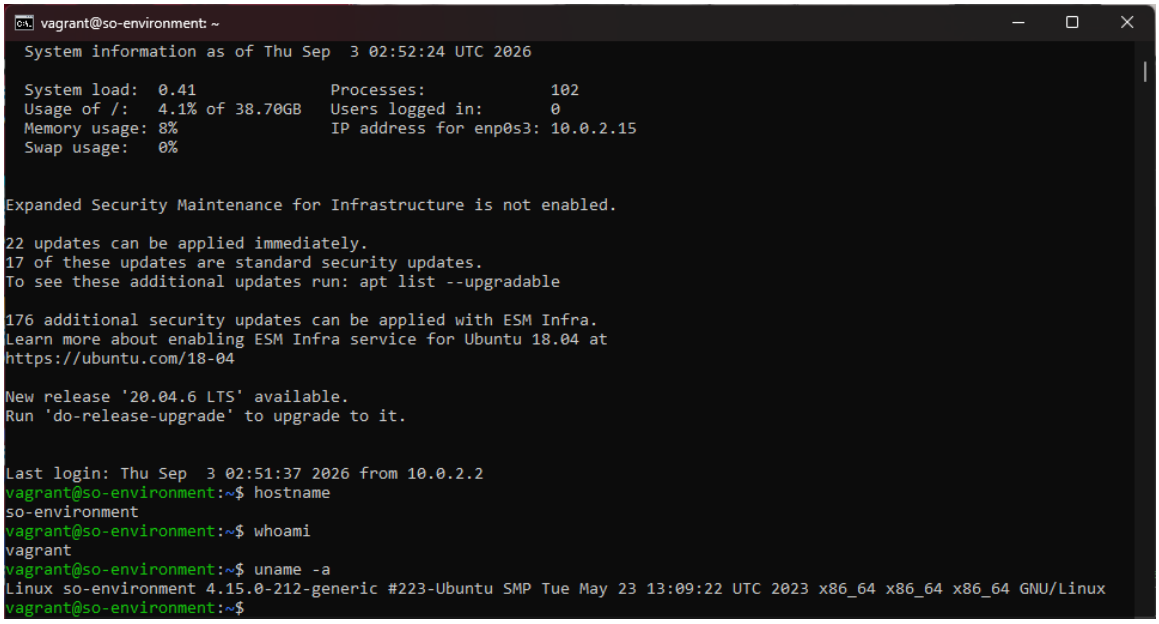


Figura 9. Sesión SSH abierta: el prompt cambia a vagrant@so-environment y uname -a confirma que es Linux.

El prompt pasó de `PS C:\Users\Lost\laboratorio-1>` a `vagrant@so-environment:~$`. El banner de Ubuntu dice 18.04, 102 procesos y 1.9 GB de memoria, y `uname -a` devuelve el kernel 4.15.0-212-generic sobre x86_64. Ya estoy dentro de otro sistema operativo, corriendo dentro del mío.

Paso 9. Cómo funciona esa conexión

Me dio curiosidad qué está pasando por debajo, así que salí y le pregunté a Vagrant:

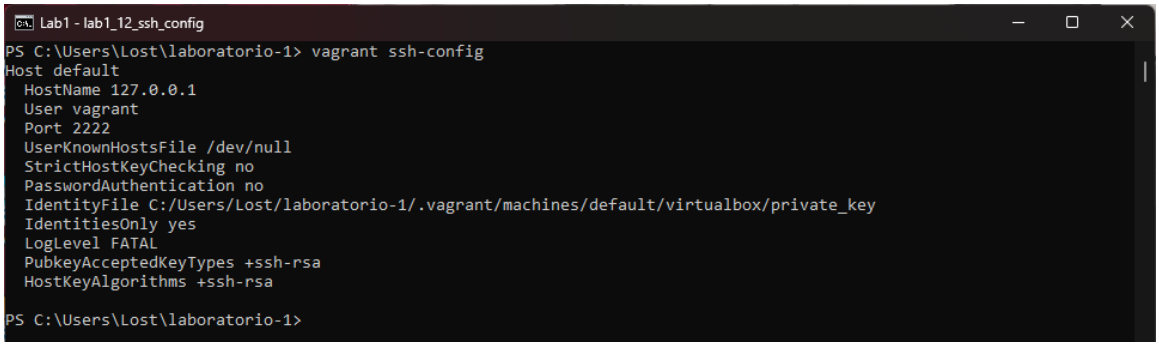


Figura 10. vagrant ssh-config muestra la configuración real de la conexión.

La conexión no va a la IP de la máquina virtual: va a `127.0.0.1` en el puerto `2222`. VirtualBox tiene una regla de redirección que agarra lo que llegue a ese puerto del anfitrión y lo manda al puerto 22 del huésped. Por eso funciona con una red NAT y sin configurar nada. La autenticación es por llave, con `PasswordAuthentication no`, y la llave está dentro de la carpeta oculta `.vagrant` del propio laboratorio.

Verificación del entorno

Paso 10. El programa de prueba

Dentro de la VM, a la carpeta que creó el aprovisionamiento:

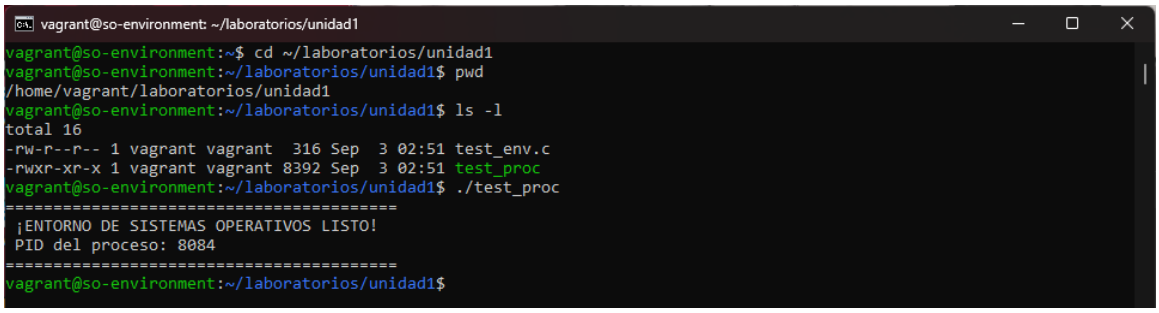


Figura 11. Ejecución de test_proc dentro de ~/laboratorios/unidad1.

Están el fuente `test_env.c` y el binario `test_proc` con permiso de ejecución (`-rwxr-xr-x`), los dos del usuario vagrant. El programa imprime el marco y su PID, que en esta corrida fue 8084. Ese número cambia en cada ejecución porque el kernel asigna un identificador nuevo a cada proceso que nace, y el programa lo pide con `getpid()`. Es un detalle pequeño pero es justo el tema del curso.

Paso 11. gcc y make

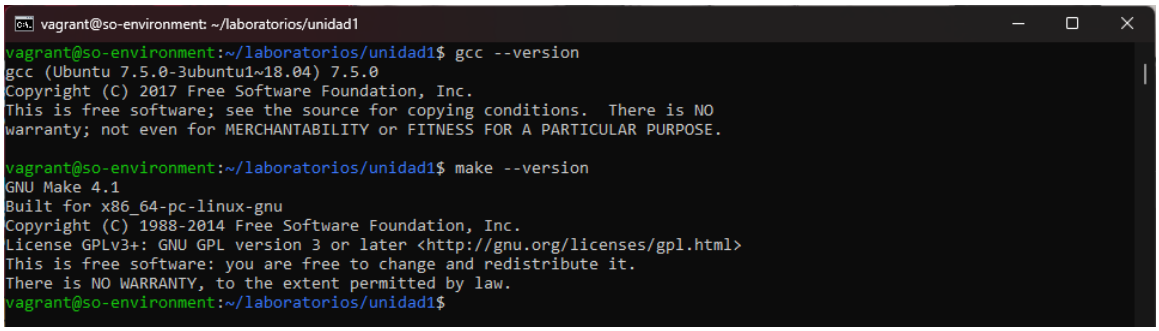


Figura 12. Versiones de gcc y make instaladas por el aprovisionamiento.

Paso 12. strace y htop

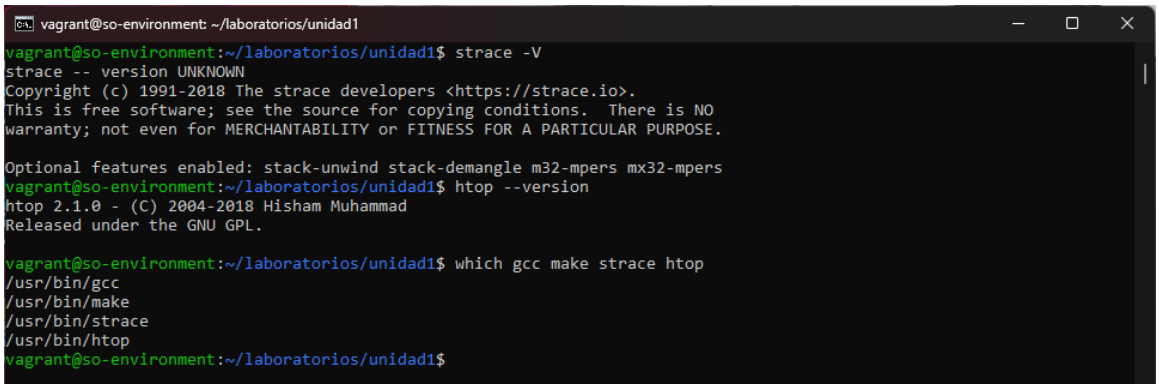


Figura 13. Versiones de strace y htop, y la ruta de los cuatro binarios.

Agregué `which gcc make strace htop` para dejar constancia de que los cuatro están en `/usr/bin` y no son alias ni funciones del shell. La versión de strace sale como `UNKNOWN`: es un detalle conocido del empaquetado de Ubuntu 18.04, el binario funciona igual pero no trae la cadena de versión compilada dentro.

Resumen de las cuatro herramientas que pide la guía:

Herramienta	Comando	Versión encontrada	Para qué sirve en el curso
gcc	gcc --version	7.5.0 (Ubuntu 7.5.0-3ubuntu1~18.04)	Compilar los programas en C de los laboratorios
make	make --version	GNU Make 4.1	Automatizar la compilación con Makefile
strace	strace -V	(cadena UNKNOWN, binario en /usr/bin/strace)	Ver las llamadas al sistema que hace un proceso
htop	htop --version	2.1.0	Monitorear procesos, CPU y memoria en vivo

Paso 13. htop en funcionamiento

Comprobar la versión demuestra que el programa está, pero no que sirva. Lo abrí:

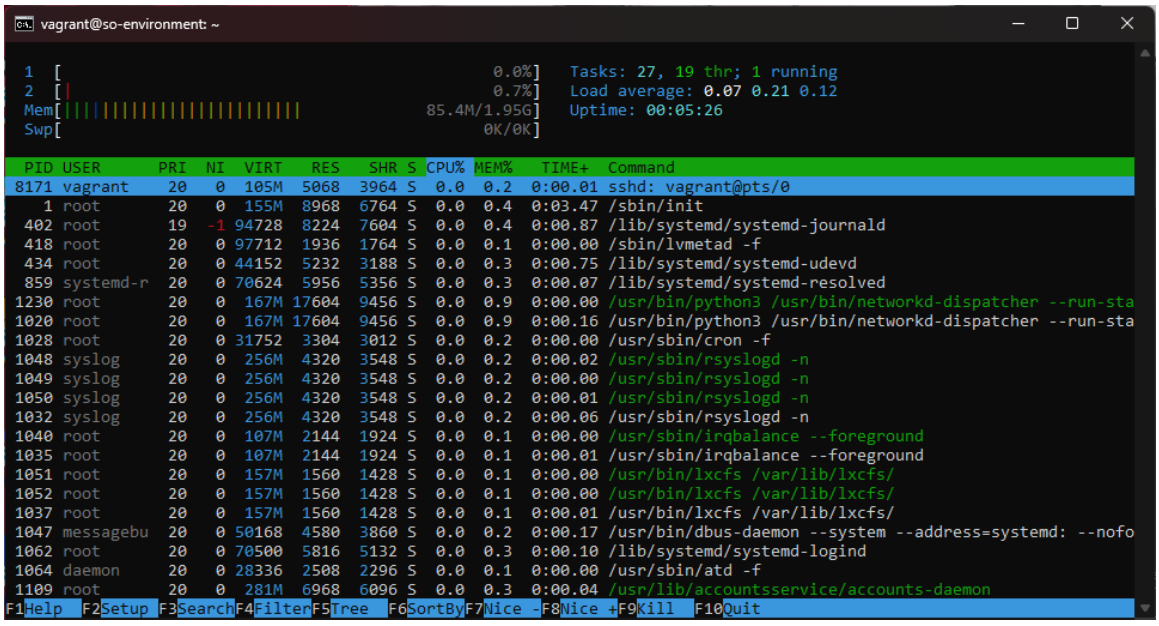


Figura 14. htop corriendo dentro de la máquina virtual.

Se ven las dos CPU asignadas, 85,4 MB usados de 1,95 GB, 27 tareas con 19 hilos y un uptime de cinco minutos. La primera fila resaltada es `sshd: vagrant@pts/0`, que es mi propia conexión: el proceso desde el cual estoy mirando la lista aparece en la lista. Abajo están `/sbin/init` con PID 1 y los demonios del sistema.

Estado final de la máquina virtual

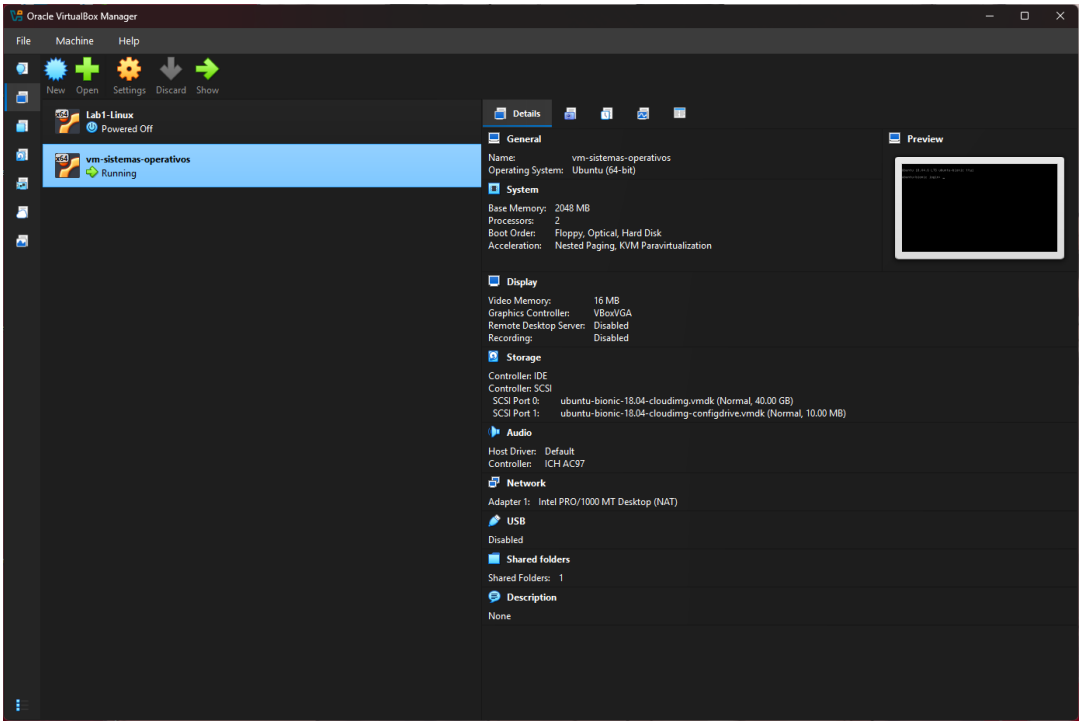


Figura 15. La VM vm-sistemas-operativos en ejecución vista desde VirtualBox Manager.

Lista de comprobación

Elemento	Comando	Resultado	Evidencia
Máquina virtual	vagrant up	Creada y aprovisionada	Figura 7
Plan de ejecución	terraform plan	1 to add, 0 to change, 0 to destroy	Figura 4
Acceso a la VM	vagrant ssh	Sesión abierta como vagrant@so-environment	Figura 9
GCC	gcc --version	7.5.0	Figura 12
Make	make --version	GNU Make 4.1	Figura 12
Strace	strace -V	Instalado en /usr/bin/strace	Figura 13
Htop	htop --version	2.1.0	Figuras 13 y 14
Programa de prueba	./test_proc	Ejecuta e imprime su PID	Figura 11

Conclusiones

El entorno quedó armado y responde: la VM arranca, se entra por SSH sin contraseña, las cuatro herramientas están instaladas y `test_proc` compila y corre. Ya se puede usar en los laboratorios que vienen.

Lo que más me llevo, sin embargo, es el intento fallido. Terraform y Vagrant describen la máquina casi con las mismas palabras (dos CPU, 2 GB, Ubuntu Bionic) y sin embargo uno funciona y el otro

no. La diferencia está en algo que ninguno de los dos archivos menciona: Terraform necesita que el huésped le conteste desde adentro a través de las Guest Additions, y Vagrant no. Un supuesto invisible sobre el software que corre dentro de la máquina fue suficiente para tumbar todo el despliegue. La infraestructura como código no elimina las dependencias, solo las esconde mejor.

También me quedó claro por qué un laboratorio se monta en una máquina virtual y no en el equipo de uno. Puedo instalar lo que sea ahí adentro, romperlo entero, borrarlo con `vagrant destroy` y volver a tener el mismo entorno limpio en unos minutos, sin tocar Windows. Y como el `Vagrantfile` es un archivo de texto de doce líneas, cualquiera con esas doce líneas termina con exactamente la misma máquina que yo. Eso vale más que la VM en sí.

Y una cosa que no esperaba: `vagrant ssh-config` me mostró que la conexión va a `127.0.0.1:2222` y no a ninguna IP de la máquina virtual. Le venía diciendo "conectarme a la VM" a algo que en realidad es una redirección de puertos del anfitrión hacia el huésped. Está bien saber que la comodidad de que todo funcione a la primera esconde una decisión de configuración que alguien tomó por mí.

Anexos

Anexo A. Comandos ejecutados

Paso	Dónde	Comando	Para qué
1	Windows, PowerShell	winget install --id Hashicorp.Vagrant -e	Instalar Vagrant
1	Windows, PowerShell	winget install --id Hashicorp.Terraform -e	Instalar Terraform
1	Windows, PowerShell	VBoxManage --version	Comprobar VirtualBox 6.1 o superior
2	Windows, PowerShell	Get-ChildItem -Force	Ver el contenido de la carpeta del laboratorio
3	Windows, PowerShell	terraform init	Descargar el proveedor de VirtualBox
4	Windows, PowerShell	terraform plan	Revisar el plan de ejecución (paso obligatorio)
5	Windows, PowerShell	terraform apply -auto-approve	Intentar crear la VM con Terraform
5	Windows, PowerShell	VBoxManage showvminfo ... --machinereadable	Diagnosticar el fallo (GuestAdditionsRunLevel)
5	Windows, PowerShell	VBoxManage unregistervm ... --delete	Borrar la VM que quedó a medias
6	Windows, PowerShell	vagrant up	Crear y aprovisionar la VM con Vagrant
7	Windows, PowerShell	vagrant status	Confirmar que la VM está corriendo
8	Windows, PowerShell	vagrant ssh	Entrar a la máquina virtual
8	VM, bash	hostname / whoami / uname -a	Confirmar en qué máquina y con qué usuario estoy
9	Windows, PowerShell	vagrant ssh-config	Ver puerto, usuario y llave de la conexión
10	VM, bash	cd ~/laboratorios/unidad1 && pwd && ls -l	Ubicarse en la carpeta de la unidad 1
10	VM, bash	./test_proc	Ejecutar el programa de prueba
11	VM, bash	gcc --version / make --version	Verificar el compilador y make
12	VM, bash	strace -V / htop --version	Verificar strace y htop
12	VM, bash	which gcc make strace htop	Confirmar la ruta de los binarios
13	VM, bash	htop	Ver procesos, CPU y memoria en vivo
13	VM, bash	nproc / free -h / df -h /	Recursos asignados a la máquina virtual

Anexo B. Salidas de texto

Las mismas comprobaciones de las capturas, en texto, por si hace falta buscarlas o copiarlas.

```
vagrant@so-environment:~$ gcc --version | head -1
gcc (Ubuntu 7.5.0-3ubuntu1~18.04) 7.5.0
vagrant@so-environment:~$ make --version | head -1
GNU Make 4.1
vagrant@so-environment:~$ strace -V | head -1
strace -- version UNKNOWN
vagrant@so-environment:~$ htop --version | head -1
htop 2.1.0 - (C) 2004-2018 Hisham Muhammad
vagrant@so-environment:~$ which gcc make strace htop
/usr/bin/gcc
/usr/bin/make
/usr/bin/strace
/usr/bin/htop
```

```
vagrant@so-environment:~$ uname -r
4.15.0-212-generic
vagrant@so-environment:~$ ps aux | wc -l
97
```

Y la configuración de la conexión SSH que arma Vagrant:

```
C:\Users\Lost\laboratorio-1> vagrant ssh-config
Host default
  HostName 127.0.0.1
  User vagrant
  Port 2222
  UserKnownHostsFile /dev/null
  StrictHostKeyChecking no
  PasswordAuthentication no
  IdentityFile C:/Users/Lost/laboratorio-1/.vagrant/machines/default/virtualbox/private_key
  IdentitiesOnly yes
  LogLevel FATAL
```